

Práctica 3

Portal de configuración WiFi (modo AP) e integración con Ubidots en ESP32
TTGO T-Display

Samuel Gil

Octubre de 2026

1. Introducción

En esta práctica se implementó, en una placa TTGO T-Display (ESP32), un sistema que combina un portal de configuración WiFi en modo Access Point con la integración a la plataforma IoT Ubidots. Desde un navegador se ingresa la red a la que se conectará el dispositivo; una vez conectado, la placa publica la temperatura y la humedad medidas con un sensor DHT11 y recibe las órdenes de control sw1 y sw2, que se muestran en su pantalla y en un tablero web.

Funcionamiento general

- Sin red guardada: la placa crea el punto de acceso samuel_AP y sirve el portal de configuración.
- Con red guardada: se conecta a la red, publica cada 5 s y recibe los interruptores de Ubidots.
- Pulsación larga (3 s) del botón: la placa vuelve al modo de configuración.

2. Desarrollo

2.1 Portal de configuración WiFi (modo AP)

Cuando la placa no tiene una red guardada, crea el punto de acceso samuel_AP. Desde el celular se abre la dirección 192.168.4.1, se escribe el nombre y la contraseña de la red y se guardan en la EEPROM; la placa se reinicia en modo estación y muestra «Iniciando...» mientras se conecta.

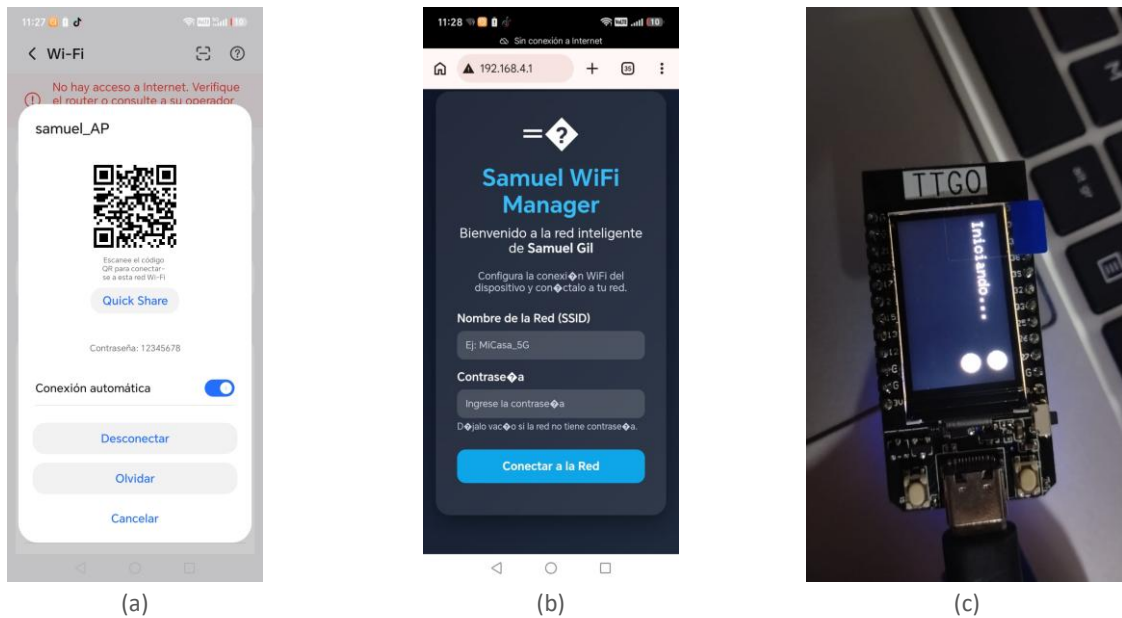


Figura 1. Modo AP: (a) celular conectado a samuel_AP, (b) portal de configuración y (c) pantalla de la placa al iniciar.

3.2 Integración con Ubidots (modo STA)

Con la red configurada, la placa publica Temperatura, Humedad y ADC cada 5 s y se suscribe a switch_1 y switch_2. En la pantalla se muestran la temperatura, la humedad y dos indicadores circulares que reflejan el estado de cada interruptor del tablero (morado para switch_1 y rojo para switch_2).

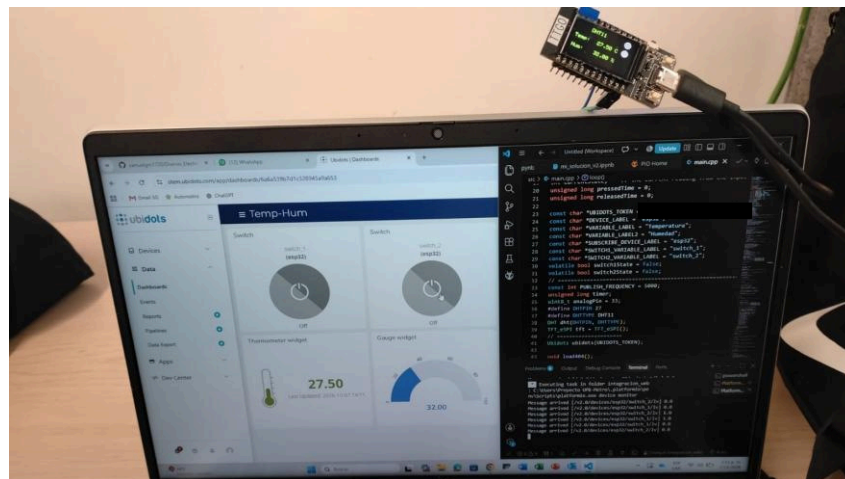


Figura 2. Placa y tablero con switch_1 y switch_2 apagados.

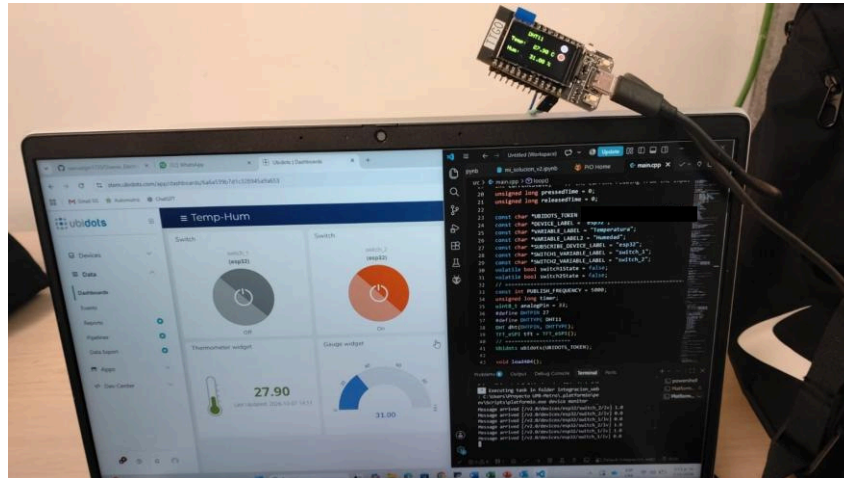


Figura 3. Placa y tablero con switch_2 encendido.

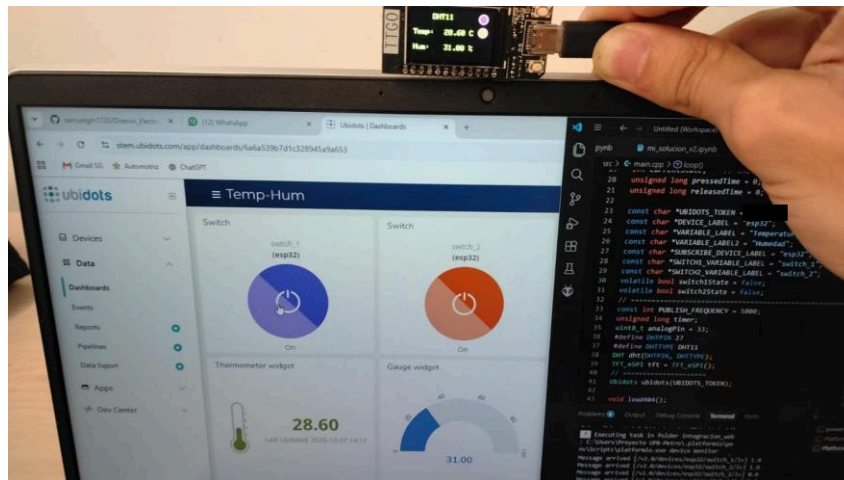


Figura 4. Placa y tablero con switch_1 y switch_2 encendidos.

En las siguientes capturas se observa el tablero de Ubidots junto con el monitor serial, donde llegan los mensajes de los interruptores.

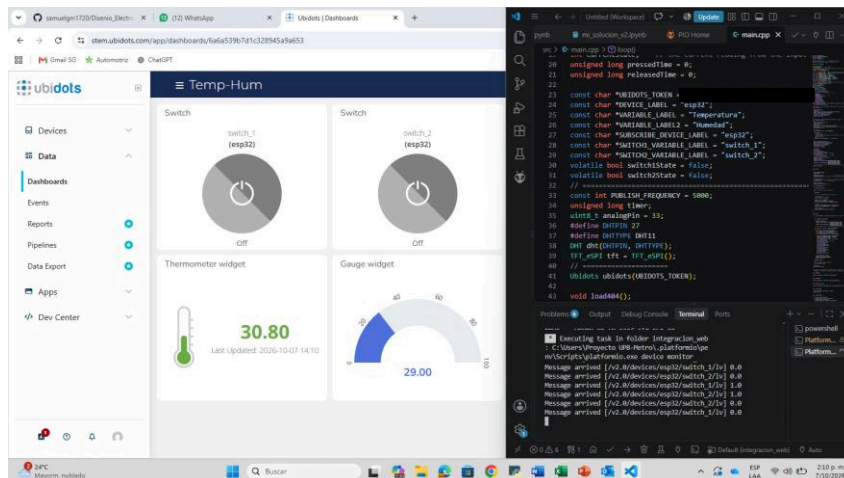


Figura 5. Tablero y monitor serial con ambos interruptores apagados.

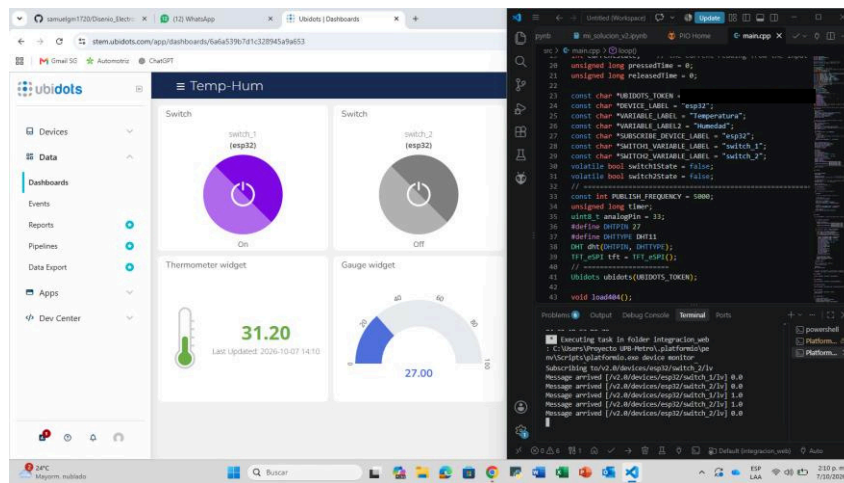


Figura 6. Tablero y monitor serial con switch_1 encendido y switch_2 apagado.

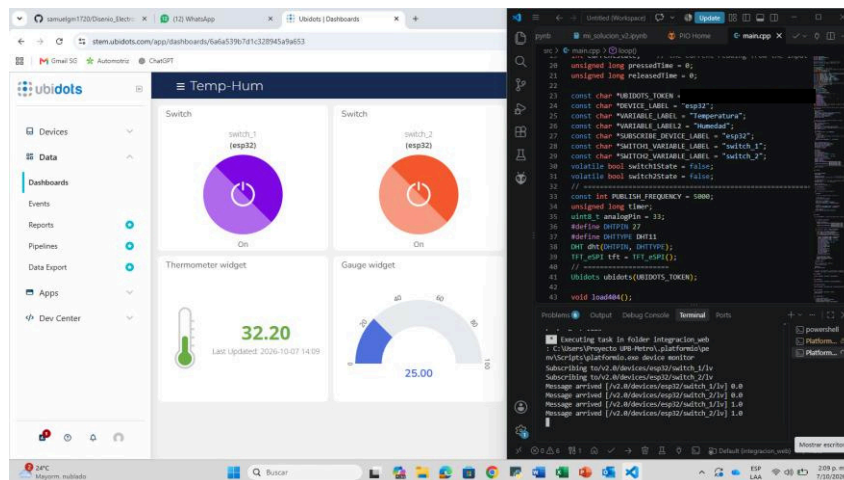


Figura 7. Tablero y monitor serial con ambos interruptores encendidos.

4. Código

```

#include <Arduino.h>
#include <WiFi.h>
#include <WebServer.h>
#include <stdlib.h>

#include "data.h"
#include "Settings.h"
#include "UbidotsEsp32Mqtt.h"
#include <DHT.h>
#include <TFT_eSPI.h>

#define BUTTON_LEFT 0          // btn activo en bajo
#define LONG_PRESS_TIME 3000 // 3000 milis = 3s

WebServer server(80);

Settings settings;
int lastState = LOW; // para el btn
int currentState;    // the current reading from the input pin
unsigned long pressedTime = 0;
unsigned long releasedTime = 0;

const char *UBIDOTS_TOKEN = [REDACTED];
const char *DEVICE_LABEL = "esp32";
const char *VARIABLE_LABEL = "Temperatura";
const char *VARIABLE_LABEL2 = "Humedad";
const char *SUBSCRIBE_DEVICE_LABEL = "esp32";
const char *SWITCH1_VARIABLE_LABEL = "switch_1";
const char *SWITCH2_VARIABLE_LABEL = "switch_2";
volatile bool switch1State = false;
volatile bool switch2State = false;
// =====
const int PUBLISH_FREQUENCY = 5000;
unsigned long timer;
uint8_t analogPin = 33;
#define DHTPIN 27
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
TFT_eSPI tft = TFT_eSPI();
// =====
Ubidots ubidots(UBIDOTS_TOKEN);

void load404();
void loadIndex();
void loadFunctionsJS();
void restartESP();
void saveSettings();
bool is_STA_mode();
void AP_mode_onRst();
void STA_mode_onRst();
void detect_long_press();
void callback(char *topic, byte *payload, unsigned int length);
void drawSwitchIndicators();

// Rutina para iniciar en modo AP (Access Point) "Servidor"

```

```

void startAP()
{
  WiFi.disconnect();
  delay(19);
  Serial.println("Starting WiFi Access Point (AP)");
  WiFi.softAP("samuel_AP", "12345678");
  IPAddress IP = WiFi.softAPIP();
  Serial.print("AP IP address: ");
  Serial.println(IP);
}

// Rutina para iniciar en modo STA (Station) "Cliente"
void start_STA_client()
{
  WiFi.softAPdisconnect(true);
  WiFi.disconnect();
  delay(100);
  Serial.println("Starting WiFi Station Mode");
  WiFi.begin((const char *)settings.ssid.c_str(), (const char *)settings.password.c_str());
  WiFi.mode(WIFI_STA);

  int cnt = 0;
  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    // Serial.print(".");
    if (cnt == 100) // Si después de 100 intentos no se conecta, vuelve a modo AP
      AP_mode_onRst();
    cnt++;
    Serial.println("attempt # " + (String)cnt);
  }

  WiFi.setAutoReconnect(true);
  Serial.println(F("WiFi connected"));
  Serial.println(F("IP address: "));
  Serial.println(WiFi.localIP());
  pressedTime = millis();
  // Rutinas de Ubidots

  dht.begin();
  tft.init();
  tft.setRotation(1);
  tft.fillScreen(TFT_BLACK);
  tft.setTextColor(TFT_WHITE, TFT_BLACK);
  tft.setTextSize(2);
  tft.drawString("Iniciando...", 10, 20);
  // =====
  ubidots.setCallback(callback);
  ubidots.setup();
  ubidots.reconnect();
  ubidots.subscribeLastValue(SUBSCRIBE_DEVICE_LABEL,
                             SWITCH1_VARIABLE_LABEL);
  ubidots.subscribeLastValue(SUBSCRIBE_DEVICE_LABEL,
                             SWITCH2_VARIABLE_LABEL);
  // =====

```

```

    timer = millis();
}

void setup()
{
    Serial.begin(115200);
    delay(2000);

    EEPROM.begin(4096);           // Se inicializa la EEPROM con su tamaño max 4KB
    pinMode(BUTTON_LEFT, INPUT_PULLUP); // btn activo en bajo

    // settings.reset();
    settings.load(); // se carga SSID y PWD guardados en EEPROM
    settings.info(); // ... y se visualizan

    Serial.println("");
    Serial.println("starting...");

    if (is_STA_mode())
    {
        start_STA_client();
    }
    else // Modo Access Point & WebServer
    {
        startAP();

        /* ===== Modo Web Server ===== */

        /* HTML sites */
        server.onNotFound(load404);

        server.on("/", loadIndex);
        server.on("/index.html", loadIndex);
        server.on("/functions.js", loadFunctionsJS);

        /* JSON */
        server.on("/settingsSave.json", saveSettings);
        server.on("/restartESP.json", restartESP);

        server.begin();
        Serial.println("HTTP server started");
    }
}

void loop()
{
    if (is_STA_mode()) // Rutina para modo Station (cliente Ubidots)
    {
        if (!ubidots.connected())
        {
            ubidots.reconnect();
            ubidots.subscribeLastValue(SUBSCRIBE_DEVICE_LABEL,
                                       SWITCH1_VARIABLE_LABEL);
        }
    }
}

```



```

        ubidots.subscribeLastValue(SUBSCRIBE_DEVICE_LABEL,
                                   SWITCH2_VARIABLE_LABEL);

        //=====
    }
    if ((millis() - timer) > PUBLISH_FREQUENCY)
    {
        float value = analogRead(analogPin);
        float temperatura = dht.readTemperature();
        float humedad = dht.readHumidity();
        if (!isnan(temperatura) && !isnan(humedad))
        {
            ubidots.add(VARIABLE_LABEL, temperatura);
            ubidots.add(VARIABLE_LABEL2, humedad);
            tft.fillScreen(TFT_BLACK);
            tft.setTextColor(TFT_GREEN, TFT_BLACK);
            tft.setTextSize(2);
            tft.drawString("DHT11", 65, 10);
            tft.setCursor(10, 50);
            tft.print("Temp:");
            tft.setCursor(100, 50);
            tft.print(temperatura);
            tft.print(" C");
            tft.setCursor(10, 90);
            tft.print("Hum:");
            tft.setCursor(100, 90);
            tft.print(humedad);
            tft.print(" %");
            drawSwitchIndicators();
        }
        // =====
        ubidots.add("ADC", value);
        ubidots.publish(DEVICE_LABEL);
        timer = millis();
    }
    ubidots.loop();
}

else // rutina para AP + WebServer
    server.handleClient();

delay(10);
detect_long_press();
}

// funciones para responder al cliente desde el webserver:
// load404(), loadIndex(), loadFunctionsJS(), restartESP(), saveSettings()

void load404()
{
    server.send(200, "text/html", data_get404());
}

void loadIndex()
{
    server.send(200, "text/html", data_getIndexHTML());
}

```

```

void loadFunctionsJS()
{
  server.send(200, "text/javascript", data_getFunctionsJS());
}

void restartESP()
{
  server.send(200, "text/json", "true");
  ESP.restart();
}

void saveSettings()
{
  if (server.hasArg("ssid"))
    settings.ssid = server.arg("ssid");
  if (server.hasArg("password"))
    settings.password = server.arg("password");

  settings.save();
  server.send(200, "text/json", "true");
  STA_mode_onRst();
}

// Rutina para verificar si ya se guardó SSID y PWD del cliente
// is_STA_mode retorna true si ya se guardaron
bool is_STA_mode()
{
  if (EEPROM.read(flagAdr))
    return true;
  else
    return false;
}

void AP_mode_onRst()
{
  EEPROM.write(flagAdr, 0);
  EEPROM.commit();
  delay(100);
  ESP.restart();
}

void STA_mode_onRst()
{
  EEPROM.write(flagAdr, 1);
  EEPROM.commit();
  delay(100);
  ESP.restart();
}

void detect_long_press()
{
  // read the state of the switch/button:
  currentState = digitalRead(BUTTON_LEFT);
}

```

```

    if (lastState == HIGH && currentState == LOW) // button is pressed
        pressedTime = millis();
    else if (lastState == LOW && currentState == HIGH)
    { // button is released
        releasedTime = millis();

        // Serial.println("releasedtime" + (String)releasedTime);
        // Serial.println("pressedtime" + (String)pressedTime);

        long pressDuration = releasedTime - pressedTime;

        if (pressDuration > LONG_PRESS_TIME)
        {
            Serial.println("(Hard reset) returning to AP mode");
            delay(500);
            AP_mode_onRst();
        }
    }

    // save the the last state
    lastState = currentState;
}

void drawSwitchIndicators()
{
    int cx1 = 210, cy1 = 20, r = 12;
    tft.fillCircle(cx1, cy1, r, switch1State ? TFT_PURPLE : TFT_DARKGREY);
    tft.drawCircle(cx1, cy1, r, TFT_WHITE);
    int cx2 = 210, cy2 = 55;
    tft.fillCircle(cx2, cy2, r, switch2State ? TFT_RED : TFT_DARKGREY);
    tft.drawCircle(cx2, cy2, r, TFT_WHITE);
}

void callback(char *topic, byte *payload, unsigned int length)
{
    Serial.print("Message arrived [");
    Serial.print(topic);
    Serial.print("] ");
    char payloadStr[length + 1];
    for (unsigned int i = 0; i < length; i++)
    {
        payloadStr[i] = (char)payload[i];
        Serial.print((char)payload[i]);
    }
    payloadStr[length] = '\0';
    Serial.println();
    float value = atof(payloadStr);
    String topicStr = String(topic);
    if (topicStr.indexOf(SWITCH1_VARIABLE_LABEL) != -1)
    {
        switch1State = (value == 1);
        drawSwitchIndicators();
    }
    else if (topicStr.indexOf(SWITCH2_VARIABLE_LABEL) != -1)
    {

```

```
    switch2State = (value == 1);  
    drawSwitchIndicators();  
  }  
}
```